CORELINK — PLANO DE INCREMENTO FUNCIONAL • NATIVIDADE-TO

# VULNERABILIDADE 2: ATAQUE DE DUPLO CLIQUE / CONCORRÊNCIA DE CHECKOUT

HJ Infor — Bloqueio Síncrono (checkoutLockRef + isLoading), Token de Idempotência e Mutex Anti-TOCTOU em /checkout

| MÓDULO DE SEGURANÇA | CATEGORIA OWASP | STACK | DATA DE EXECUÇÃO |
|---|---|---|---|
| Faturamento B2B (/checkout) | A04:2021 — Race Condition | React 19, Mutex, Prisma ACID | 25/10/2026 |

## 1. VULNERABILIDADE SELECIONADA E PRIORIZAÇÃO

VULNERABILIDADE MITIGADA

**Vulnerabilidade 2 (Ataque de Duplo Clique / Concorrência de Checkout):** No formulário de faturamento (`/checkout`), o botão permanecia interativo durante a requisição assíncrona, permitindo múltiplos disparos simultâneos.

**Justificativa Técnica:** Em operações de faturamento e emissão de pedidos B2B, a janela entre o primeiro clique do usuário e a resposta da requisição assíncrona (ou o intervalo de microssegundos anterior à re-renderização do DOM pelo React) permitia que cliques repetidos disparassem execuções concorrentes de `createOrder`. Além disso, em um ataque de concorrência direta (*Race Condition / Time-of-Check to Time-of-Use — TOCTOU*), duas requisições que chegassem ao servidor no mesmo milissegundo podiam ultrapassar uma verificação simples de leitura e gravar pedidos duplicados ou consumir duas vezes um cupom de uso único.

## 2. RESULTADO ESPERADO E COMPORTAMENTO ALCANÇADO

MUTEX ANTI-TOCTOU ATIVO

O fluxo de checkout (`CheckoutForm.tsx` e `src/actions/order.ts`) recebeu **4 barreiras de proteção contra concorrência**: (1) **Trava Síncrona (`checkoutLockRef`)** que neutraliza cliques em menos de 1ms; (2) **Controle de Estado (`isLoading`)** que desabilita botão, cupom e checkbox LGPD com `disabled` e `pointer-events-none`; (3) **Chave de Idempotência (`idempotencyKey`)** única por sessão de checkout; e (4) **Mutex em Memória (`activeCheckoutLocks`) + Incremento Condicional Atômico (`tx.coupon.updateMany`)** no servidor, garantindo que rajadas paralelas no mesmo milissegundo abram apenas 1 transação no PostgreSQL.

## 3. PLANO DE EXECUÇÃO DETALHADO

| ATIVIDADE | RESPONSÁVEL | TECNOLOGIAS | PERÍODO | STATUS |
|---|---|---|---|---|
| **Bloqueio Síncrono e Visual no Formulário (`CheckoutForm.tsx`).** Adição de `checkoutLockRef`, `isLoading` e desativação completa dos controles durante a requisição. | Marques Ramirez | React 19 (`useState`, `useRef`), Tailwind CSS | 22/10 a 23/10/2026 | ✓ Concluída |
| **Token Criptográfico de Idempotência (`idempotencyKey`).** Geração de chave única por tentativa em `/checkout` e cache de 60s em `processedIdempotencyKeys`. | Márcio Medrado | TypeScript, Next.js Server Actions, In-Memory Cache | 23/10 a 24/10/2026 | ✓ Concluída |
| **Mutex Anti-TOCTOU e Cupom Atômico (`order.ts`).** Implementação de `activeCheckoutLocks` e verificação atômica `usedCount: { lt: usageLimit }`. | Marques Ramirez & Márcio Medrado | Promise Coalescing, Prisma `$transaction`, PostgreSQL | 24/10 a 25/10/2026 | ✓ Concluída |
| **Auditoria de Ataque de Concorrência (`test-concurrency-vuln2.mjs`).** Simulação de 5 cliques simultâneos na UI e 3 disparos POST paralelos via rede. | Taís Fernandes, Romildo & Vandelice | Node.js, `Promise.all`, Teste de Stress TOCTOU | 25/10/2026 | ✓ Concluída |
| **Documentação de Segurança e Relatório PDF.** Consolidação da arquitetura anti-concorrência no padrão institucional de 2 páginas A4. | Andréa Ribeiro | Git, GitHub, Markdown, PDF Engine | 25/10/2026 | ✓ Concluída |

## 4. ESPECIFICAÇÃO DAS 4 CAMADAS DE CONTENÇÃO CONTRA CONCORRÊNCIA NO CHECKOUT

DEFESA EM PROFUNDIDADE

| CAMADA / MECANISMO | LOCALIZAÇÃO NO CÓDIGO | PAPEL NA NEUTRALIZAÇÃO DO ATAQUE DE DUPLO CLIQUE E RACE CONDITION |
|---|---|---|
| **1. Trava Síncrona + Estado UI** (`checkoutLockRef` & `isLoading`) | `CheckoutForm.tsx` (Frontend React 19) | Intercepta cliques repetidos em < 1ms antes da re-renderização e aplica `disabled` + `pointer-events-none` ao botão *"Confirmar Pedido"*. |
| **2. Token de Idempotência** (`idempotencyKeyRef`) | `CheckoutForm.tsx` ➔ `src/actions/order.ts` | Associa um identificador único (`chk_prod_time_nonce`) à tentativa; retransmissões de rede recebem o mesmo `orderId` sem reprocessar. |
| **3. Mutex em Voo (Anti-TOCTOU)** (`activeCheckoutLocks`) | `src/actions/order.ts` (Server Action) | Se 2 ou mais requisições paralelas chegarem no mesmo milissegundo (0ms), coalesce todas na 1ª promessa ativa sem abrir nova transação. |
| **4. Guarda Atômica no Banco** (`duplicateWindow` & `updateMany`) | `src/actions/order.ts` (Prisma ORM / PostgreSQL) | Impede criação de pedido `PENDING` repetido na janela de 15s e incrementa cupons atomicamente (`usedCount < usageLimit`). |

## 5. EVIDÊNCIA VISUAL: NEUTRALIZAÇÃO DE DUPLO CLIQUE NA UI E RACE CONDITION NO SERVIDOR

AUDITORIA TOCTOU

| RAJADA NA UI (/CHECKOUT) | CLIQUES BLOQUEADOS NA UI | ATAQUE POST PARALELO (REDE) | PEDIDOS DUPLICADOS NO DB |
|---|---|---|---|
| 5 Cliques (0ms) | 4 / 4 Retidos (100%) | 3 Reqs ➔ 1 Transação | 0 Duplicações |

**1. Contenção no Cliente (CheckoutForm.tsx)**

Comportamento durante rajada de 5 cliques consecutivos no botão

- Clique #1 (0ms): checkoutLockRef=true | isLoading=true — ✓ Enviado
- Cliques #2 e #3 (+1ms): Interceptados por checkoutLockRef — ✗ Bloqueados
- Cliques #4 e #5 (+16ms): Botão disabled + pointer-events-none — ✗ Bloqueados

**2. Contenção no Servidor contra Bypass (src/actions/order.ts)**

3 requisições HTTP POST disparadas simultaneamente em t = 0ms

- POST #1 (0ms): Registra activeCheckoutLocks.set(lockKey) — ✓ 1ª Transação DB
- POST #2 (0ms): Detecta Mutex em voo ➔ Aguarda POST #1 — ↻ Coalescido
- POST #3 (+20ms): Interceptado por idempotencyKey / 15s Window — ✓ Deduplicado

## 6. IMPLEMENTAÇÃO TÉCNICA (SRC/ACTIONS/ORDER.TS & CHECKOUTFORM.TSX) E LOG DE AUDITORIA

MUTEX + PRISMA ACID

**6.1 Mutex Anti-TOCTOU e Cupom Atômico (src/actions/order.ts)**

```
export async function createOrder(
  productId: string, couponId?: string, idempotencyKey?: string
): Promise<CheckoutResult> {
  const userId = session.user.id as string;
  const lockKey = `${userId}:${productId}`;

  if (idempotencyKey && processedIdempotencyKeys.has(idempotencyKey)) {
    return { success: true, orderId: processedIdempotencyKeys.get(idempotencyKey)!.orderId, deduplicated: true };
  }
  // Mutex contra Race Condition (TOCTOU) em chamadas simultâneas (0ms)
  const inFlightPromise = activeCheckoutLocks.get(lockKey);
  if (inFlightPromise) {
    const firstResult = await inFlightPromise;
    return { ...firstResult, deduplicated: true };
  }
  // Incremento atômico condicional dentro de prisma.$transaction:
  const updated = await tx.coupon.updateMany({
    where: { id: coupon.id, isActive: true, usedCount: { lt: coupon.usageLimit } },
    data: { usedCount: { increment: 1 } },
  });
  if (updated.count === 0) throw new Error("COUPON_INVALID");
}
```

**6.2 Log Real da Auditoria (scripts/test-concurrency-vuln2.mjs)**

```
======================================================================
 AUDITORIA DE SEGURANCA - VULNERABILIDADE 2 (CONCORRENCIA DE CHECKOUT)
======================================================================
 [CLIENT][ENVIADO]   Clique #1 aceito → Disparando Server Action...
 [SERVER][PRISMA TX] Req #1 abriu transacao unica no PostgreSQL...
 [CLIENT][BLOQUEADO] Clique #2 neutralizado na UI (isLoading=true)
 [CLIENT][BLOQUEADO] Clique #3 neutralizado na UI (isLoading=true)
 [CLIENT][BLOQUEADO] Clique #4 neutralizado na UI (isLoading=true)
 [CLIENT][BLOQUEADO] Clique #5 neutralizado na UI (isLoading=true)
----------------------------------------------------------------------
 SIMULACAO DE ATAQUE DIRETO VIA REDE (BYPASS DE UI - 3 POSTS PARALELOS)
----------------------------------------------------------------------
 [SERVER][PRISMA TX] Req #1 abriu transacao unica no PostgreSQL...
 [SERVER][MUTEX TOCTOU] Req #2 chegou em paralelo (0ms) → Coalescida!
 [SERVER][MUTEX TOCTOU] Req #3 chegou em paralelo (0ms) → Coalescida!
======================================================================
 RESUMO: Cliques UI Bloqueados: 4/4 | Transacoes DB: 1 (Coalescidas: 2)
======================================================================
```

## 7. ROTEIRO DE VALIDAÇÃO E TESTES DE SEGURANÇA (HOMOLOGAÇÃO)

| CENÁRIO DE TESTE | PROCEDIMENTO EXECUTADO | RESULTADO OBTIDO NA HOMOLOGAÇÃO |
|---|---|---|
| **Ataque de Duplo Clique Rápido em `/checkout`** | Executar 5 cliques rápidos no botão *"Confirmar Pedido"* após marcar o aceite LGPD. | O 1º clique acionou `checkoutLockRef` e desabilitou o botão (*"Processando Pedido..."*); os 4 cliques seguintes foram bloqueados na UI. |
| **Disparo Concorrente via Rede (TOCTOU 0ms)** | Disparar 3 requisições paralelas (`Promise.all`) direto para `createOrder` no mesmo milissegundo. | O mutex `activeCheckoutLocks` interceptou as requisições #2 e #3 em voo, abrindo **apenas 1 transação** no PostgreSQL. |
| **Retransmissão com Mesma `idempotencyKey`** | Reenviar requisição com o mesmo token `idempotencyKeyRef` após a conclusão do pedido. | O cache `processedIdempotencyKeys` e a janela de 15s retornaram o pedido existente (`deduplicated: true`). |
| **Concorrência em Cupom de Uso Único** | Submeter simultaneamente duas compras utilizando cupom com `usageLimit = 1`. | A cláusula atômica `updateMany({ usedCount: { lt: usageLimit } })` garantiu que o limite jamais fosse ultrapassado. |